

▼ Lecture 9: Transfer learning

We provide a colab with training, inference, and clustering code using transfer learning models

Notebook steps:

1. Load and preprocess the dataset
2. Define the Transformer model
3. Train the model
4. Evaluate the model

```
!pip install datasets[audio]==3
!pip install matplotlib
!pip install torch
!pip install torchcodec
!pip install tqdm
!pip install transformers
!pip install scikit-learn
!pip install omar_rq
```

```
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn==1.6.0)
Requirement already satisfied: omar_rq in /usr/local/lib/python3.12/dist-packages (0.2.1)
Requirement already satisfied: lightning<3.0,>=2.3 in /usr/local/lib/python3.12/dist-packages (from omar_rq) (2.12.0)
Requirement already satisfied: torch<3.0,>=2.6 in /usr/local/lib/python3.12/dist-packages (from omar_rq) (2.8.0)
Requirement already satisfied: torchaudio<3.0,>=2.6 in /usr/local/lib/python3.12/dist-packages (from omar_rq) (2.8.0)
Requirement already satisfied: gin-config<1.0,>=0.5 in /usr/local/lib/python3.12/dist-packages (from omar_rq) (0.5.0)
Requirement already satisfied: einops<1.0,>=0.8 in /usr/local/lib/python3.12/dist-packages (from omar_rq) (0.8.0)
Requirement already satisfied: transformers<5.0,>=4.50 in /usr/local/lib/python3.12/dist-packages (from omar_rq) (4.50.0)
Requirement already satisfied: PyYAML<8.0,>=5.4 in /usr/local/lib/python3.12/dist-packages (from lightning<3.0,>=2.3) (7.0.0)
Requirement already satisfied: fsspec<2027.0,>=2022.5.0 in /usr/local/lib/python3.12/dist-packages (from fsspec==2025.10.0) (2025.10.0)
Requirement already satisfied: lightning-utilities<2.0,>=0.10.0 in /usr/local/lib/python3.12/dist-packages (from lightning<3.0,>=2.3) (0.10.1)
Requirement already satisfied: packaging<27.0,>=20.0 in /usr/local/lib/python3.12/dist-packages (from lightning<3.0,>=2.3) (25.0)
Requirement already satisfied: torchmetrics<3.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from lightning<3.0,>=2.3) (1.5.0)
Requirement already satisfied: tqdm<6.0,>=4.57.0 in /usr/local/lib/python3.12/dist-packages (from lightning<3.0,>=2.3) (4.67.1)
Requirement already satisfied: typing-extensions<6.0,>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from lightning<3.0,>=2.3) (4.12.0)
Requirement already satisfied: pytorch-lightning in /usr/local/lib/python3.12/dist-packages (from lightning<3.0,>=2.3) (2.3.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (3.16.1)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (75.8.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (1.13.3)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (3.5)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (3.1.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (12.6.77)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (12.6.77)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (12.6.80)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (12.6.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (11.3.0.4)
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (10.3.7.77)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (11.7.1.2)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (0.7.1)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (12.6.4.1)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (2.27.3)
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (12.6.77)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (12.6.85)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (1.11.1.6)
Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.6->omar_rq) (3.4.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from transformers<5.0,>=4.50) (0.34.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers<5.0,>=4.50) (2.0.2)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers<5.0,>=4.50) (2024.11.6)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers<5.0,>=4.50) (2.32.3)
Requirement already satisfied: tokenizers<0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers<5.0,>=4.50) (0.22.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers<5.0,>=4.50) (0.5.2)
Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec==2025.10.0) (4.10.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0) (1.1.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2) (3.0.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (2025.10.1)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp) (2.5.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp) (25.4.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp) (6.3.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp) (0.3.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp) (1.20.0)
```

```
import math
import random
```

```

from pathlib import Path

import torch
import torch.nn as nn
import torch.nn.functional as F
import torchaudio
import numpy as np
import matplotlib.pyplot as plt
from datasets import load_dataset
from sklearn.preprocessing import OneHotEncoder
from sklearn.metrics import accuracy_score
from torch.utils.data import DataLoader, Dataset
from torchaudio.transforms import MelSpectrogram, Resample
from tqdm.auto import tqdm, trange
from omar_rq import get_model

device = "cuda" # Configure depending on the runtime

```

```

# Use Datasets to load GTZAN
ds = load_dataset("sanchit-gandhi/gtzan", split="train")

# make random train/val splits
splits = ds.train_test_split(test_size=0.3, seed=42)
train_ds = splits["train"]
val_test_ds = splits["test"]

splits = val_test_ds.train_test_split(test_size=0.5, seed=42)
val_ds = splits["train"]
test_ds = splits["test"]

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/t>)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

```

class GTZANAUDIODataset(Dataset):
    """Audio GTZAN Genre dataset based on HuggingFace Datasets
    """

    def __init__(self, dataset, device):
        self.orig_sample_rate = 32000
        self.sample_rate = 24000
        self.audio_duration = 3 # Segment durations
        self.device = device

        self.dataset = dataset
        self.n_samples = self.sample_rate * self.audio_duration
        self.resample = Resample(orig_freq=self.orig_sample_rate, new_freq=self.sample_rate)

        # Load audio and labels
        self.audio = {}
        self.natural_labels = {}
        for track in tqdm(self.dataset, desc="Loading audio"):
            tid = Path(track["file"]).stem
            genre = tid.split(".")[0]

            x = track["audio"]["array"]
            x = torch.tensor(x)
            sr = track["audio"]["sampling_rate"]

            if sr != self.sample_rate:
                if sr != self.orig_sample_rate:
                    self.resample = Resample(orig_freq=sr, new_freq=self.sample_rate)
                    self.orig_sample_rate = sr

            x = self.resample(x.float())

            self.audio[tid] = x.to(self.device)
            self.natural_labels[tid] = genre

        self.tids = list(self.audio.keys())

        # One hot encode labels
        natural_labels = np.array(list(self.natural_labels.values())).reshape(-1, 1)
        ohe = OneHotEncoder()
        one_hot_labels = ohe.fit_transform(natural_labels).toarray()
        self.labels = {k: v for k, v in zip(self.tids, one_hot_labels)}

    def __len__(self):

```

```

        return len(self.tids)

    def get_start_end_indices(self, min, max):
        start = random.randint(min, max - self.n_samples)
        end = start + self.n_samples
        return start, end

    def __getitem__(self, idx):
        tid = self.tids[idx]
        audio_full = self.audio[tid]
        labels = self.labels[tid]
        natural_labels = self.natural_labels[tid]

        # get the segment from a random position in the track.
        min_pos, max_pos = 0, audio_full.shape[0]

        start, end = self.get_start_end_indices(min_pos, max_pos)
        audio = audio_full[start:end]

        return {"audio": audio, "labels": labels, "natural_labels": natural_labels}

class GTZANOMARRQDataset(Dataset):
    """OMAR-RQ embedding dataset based on GTZAN audio.
    """

    def __init__(self, audio_dataset, device):
        self.device = device
        self.audio_dataset = audio_dataset

        self.labels = self.audio_dataset.labels
        self.natural_labels = self.audio_dataset.natural_labels
        self.tids = self.audio_dataset.tids

        # Instantiate OMAR-RQ model
        self.model = get_model(model_id="mtg-upf/clap_omarrq_mp_small_music", device=self.device)
        self.model.eval()

        # Select intermediate layer
        self.layers = [6]

        # Load audio and labels
        self.embeddings = {}
        for id, audio in tqdm(self.audio_dataset.audio.items(), desc="Computing embeddings"):

            audio = torch.tensor(audio).unsqueeze(0)
            audio = audio.float()
            audio.to(self.device)

            with torch.no_grad():
                embeddings = self.model.extract_embeddings(audio, layers=self.layers)

            embeddings = torch.mean(embeddings, dim=2)
            embeddings = embeddings.squeeze()

            self.embeddings[id] = embeddings

    def __len__(self):
        return len(self.tids)

    def __getitem__(self, idx):
        tid = self.tids[idx]
        embeddings = self.embeddings[tid]
        labels = self.labels[tid]
        natural_labels = self.natural_labels[tid]

        return {"embeddings": embeddings, "labels": labels, "natural_labels": natural_labels}

# Audio datasets
train_audio_dataset = GTZANAUDIODataset(train_ds, device=device)
val_audio_dataset = GTZANAUDIODataset(val_ds, device=device)
test_audio_dataset = GTZANAUDIODataset(test_ds, device=device)

# Embedding datasets
train_dataset = GTZANOMARRQDataset(train_audio_dataset, device=device)
val_dataset = GTZANOMARRQDataset(val_audio_dataset, device=device)
test_dataset = GTZANOMARRQDataset(test_audio_dataset, device=device)

batch_size = 32

val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

```

```

OMAR-RQ: 396 weights loaded for `net`
OMAR-RQ: 2 weights loaded for `embedding_layer`
Computing embeddings: 100% 699/699 [00:23<00:00, 30.56it/s]
/tmp/ipython-input-3368046715.py:90: UserWarning: To copy construct from a tensor, it is recommended to use source tensor
  audio = torch.tensor(audio).unsqueeze(0)
OMAR-RQ: 396 weights loaded for `net`
OMAR-RQ: 2 weights loaded for `embedding_layer`
Computing embeddings: 100% 150/150 [00:04<00:00, 30.82it/s]
OMAR-RQ: 396 weights loaded for `net`
OMAR-RQ: 2 weights loaded for `embedding_layer`
Computing embeddings: 100% 150/150 [00:06<00:00, 22.35it/s]

```

```

class FeatureExtractor(nn.Module):
    """Log mel-spectrogram feature extractor.
    """

    def __init__(self, **kwargs):
        super().__init__()
        self.melspectrogram = MelSpectrogram(**kwargs)

    def forward(self, x):
        x = self.melspectrogram(x)

        # Log-compression
        x = torch.log10(1 + 1000 * x)
        return x

```

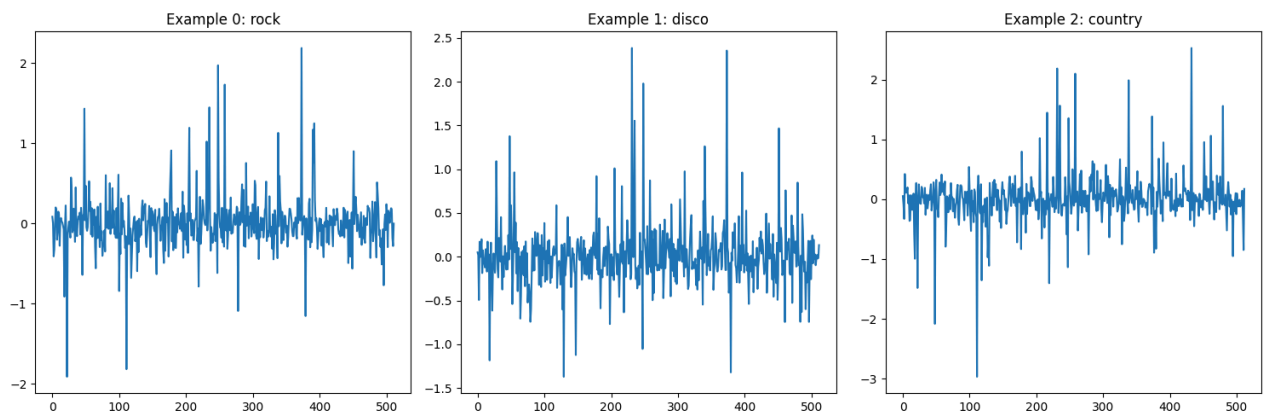
```

# Fetch a batch of data
batch = next(iter(val_loader))
embeddings = batch["embeddings"].cpu().numpy()

# Visualize the first three mel-spectrograms in the batch
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for i in range(3):
    axes[i].plot(embeddings[i])
    axes[i].set_title(f"Example {i}: {batch['natural_labels'][i]}")

plt.tight_layout()
plt.show()

```



```

n_classes = len(set(val_dataset.natural_labels.values()))

print("Embeddings shape:", embeddings.shape)
print("Number of classes:", n_classes)

```

```

Embeddings shape: (32, 512)
Number of classes: 10

```

```

class Transformer(nn.Module):
    """Transformer model for audio classification.

```

```

Args:
    kernel (tuple, optional): Kernel size for the convolutional patch embedding. Defaults to (8, 8).
    stride (tuple, optional): Stride for the convolutional patch embedding. Defaults to (8, 8).
    n_blocks (int, optional): Number of Transformer encoder blocks. Defaults to 2.
    emb_dim (int, optional): Embedding dimension. Defaults to 128.
    n_heads (int, optional): Number of attention heads. Defaults to 2.
    mlp_ratio (int, optional): Ratio of MLP hidden dimension to embedding dimension. Defaults to 2.
    output_size (int, optional): Number of output classes. Defaults to 10.
.....

def __init__(
    self,
    kernel: tuple = (8, 8),
    stride: tuple = (8, 8),
    n_blocks: int = 2,
    emb_dim: int = 128,
    n_heads: int = 2,
    mlp_ratio: int = 2,
    output_size: int = 10,
    pos_enc_gain: float = 0.0,
    cls_token_ratio: float = 0.0,
):
    super().__init__()

    self.n_blocks = n_blocks
    self.emb_dim = emb_dim
    self.n_heads = n_heads
    self.mlp_ratio = mlp_ratio
    self.output_size = output_size
    self.pos_enc_gain = pos_enc_gain
    self.cls_token_ratio = cls_token_ratio

    self.norm = nn.BatchNorm2d(1)

    padding = kernel[0] // 2
    self.patching = nn.Conv2d(
        1, self.emb_dim, kernel_size=kernel, stride=stride, padding=padding
    )

    self.blocks = nn.ModuleList([
        nn.TransformerEncoderLayer(
            self.emb_dim,
            self.n_heads,
            dim_feedforward=self.emb_dim * self.mlp_ratio,
            dropout=0.1,
            norm_first=True,
        ) for _ in range(self.n_blocks)])

    self.cls = nn.Parameter(torch.randn(1, 1, self.emb_dim))

    nn.init.normal_(self.cls, std=0.02)

    self.fc1 = nn.Linear(self.emb_dim, self.emb_dim)
    self.fc2 = nn.Linear(self.emb_dim, self.output_size)

def positional_encoding(self, seq_len):
    position = torch.arange(seq_len, dtype=torch.float32).unsqueeze(1)

    div_term = torch.exp(torch.arange(0, self.emb_dim, 2, dtype=torch.float32) * -(math.log(10000.0) / self.emb_dim))

    pe = torch.zeros(seq_len, self.emb_dim, device=self.cls.device)

    pe[:, 0::2] = torch.sin(position * div_term)
    pe[:, 1::2] = torch.cos(position * div_term)
    return pe

def forward(self, x):
    # Add embedding dimension
    x = x.unsqueeze(1)

    # Normalize input to zero-mean unit std.
    x = self.norm(x)

    # Divide the signal into patches
    x = self.patching(x) # [B, 1, freq, time] -> [b, E, M, T]

    B, E, M, T = x.shape

    # Generate positional encodings for frequency and time
    pe_m = self.positional_encoding(M) # (M, E)
    pe_t = self.positional_encoding(T) # (T, E)

```

```

pe_m_resaped = pe_m.transpose(0, 1).unsqueeze(0).unsqueeze(3) # (1, E, F, 1)
pe_t_resaped = pe_t.transpose(0, 1).unsqueeze(0).unsqueeze(2) # (1, E, 1, T)

# Add positional encoding with small gain.
x = x + self.pos_enc_gain * (pe_m_resaped + pe_t_resaped)

x = x.flatten(start_dim=2) # [B, E, F, T] -> [B, E, F * T]
x = x.swapaxes(1, 2) # [B, E, F * T] -> [B, F * T, E]

# Concatenate CLS token
cls = self.cls.repeat(B, 1, 1)
x = torch.cat([cls, x], dim=1)

# Transformer backbone
for block in self.blocks:
    x = block(x)

# Classification on top of the CLS + average of the rest of tokens
x_cls = x[:, 0, :]
x_mean = x[:, 1:, :].mean(dim=1)
x = self.cls_token_ratio * x_cls + (1 - self.cls_token_ratio) * x_mean

# Dense classification head
x = F.layer_norm(x, (self.emb_dim,))
x = F.relu(self.fc1(x))
x = self.fc2(x)

return x

```

```

class MLP(nn.Module):
    def __init__(
        self,
        input_size: int=512,
        dense_size: int=64,
        output_size: int=10,
    ):
        super().__init__()
        self.input_size = input_size
        self.dense_size = dense_size
        self.output_size = output_size

        self.fc1 = nn.Linear(self.input_size, self.dense_size)
        self.fc2 = nn.Linear(self.dense_size, self.output_size)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

```

✓ Step 3: Train the models

Set up the training loop with loss functions and optimizers.

```

def step(batch, loss_fn, model, optimizer=None, mode="audio"):
    y = batch["labels"].to(device)

    if mode == "audio":
        x = batch["audio"].to(device)
        x = feature_extractor(x)
    elif mode == "embeddings":
        x = batch["embeddings"]

    # Forward
    y_est = model(x)

    # Compute loss
    loss = loss_fn(y_est, y)
    step_loss = loss.item()

    # Compute metrics
    y_est = y_est.softmax(dim=1)
    y_int = torch.argmax(y, dim=1).cpu().numpy()
    y_est_int = torch.argmax(y_est, dim=1).cpu().numpy()

    step_acc = accuracy_score(y_int, y_est_int)

    # Backprop
    if optimizer:

```

```

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    return step_loss, step_acc

def train(
    model,
    train_dataset,
    val_dataset,
    n_epochs=3,
    lr=0.001,
    device="cuda",
    batch_size=32,
    mode="audio",
):
    # Use Adam optimizer
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    loss_fn = torch.nn.CrossEntropyLoss()

    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

    pbar = trange(n_epochs, desc="Training", total=n_epochs)
    for epoch in range(n_epochs):
        # Training loop

        model.train()
        total_loss, total_acc = 0, 0

        for batch in train_loader:
            step_loss, step_acc = step(batch, loss_fn, model, optimizer, mode=mode)
            total_loss += step_loss
            total_acc += step_acc

        train_loss = total_loss / len(train_loader)
        train_acc = total_acc / len(train_loader)

        # Validation loop
        model.eval()
        total_loss, total_acc = 0, 0

        for batch in val_loader:
            step_loss, step_acc = step(batch, loss_fn, model, mode=mode)
            total_loss += step_loss
            total_acc += step_acc

        val_loss = total_loss / len(val_loader)
        val_acc = total_acc / len(val_loader)

        pbar.set_postfix({
            # "epoch": f"{epoch+1}",
            "train loss": f"{train_loss:.3f}",
            "train acc": f"{train_acc:.3f}",
            "val loss": f"{val_loss:.3f}",
            "val acc": f"{val_acc:.3f}",
        })
        pbar.update(1)

def test(model, dataset, device="cuda", batch_size=32, mode="audio"):
    # Test loop
    test_loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
    loss_fn = torch.nn.CrossEntropyLoss()

    model.eval()
    total_loss, total_acc = 0, 0

    for batch in test_loader:
        step_loss, step_acc = step(batch, loss_fn, model, mode=mode)
        total_loss += step_loss
        total_acc += step_acc

    test_loss = total_loss / len(test_loader)
    test_acc = total_acc / len(test_loader)

    print(
        f"test loss: {test_loss:.3f}, "
        + f"test acc: {test_acc:.3f}"
    )

```

```
feature_extractor = FeatureExtractor(n_mels=64, n_fft=1024).to(device)

model = Transformer(
    n_blocks=2,
    emb_dim=64,
    n_heads=4,
).to(device)

print("Training Transformer encoder")

num_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Number of trainable parameters: {num_params}")

train(model, train_audio_dataset, val_audio_dataset, n_epochs=100, device=device, batch_size=32, mode="audio")

print("\n")
test(model, test_audio_dataset, mode="audio")
```

Training Transformer encoder
Number of trainable parameters: 75980

Training: 100%

100/100 [00:25<00:00, 4.11it/s, train loss=0.807, train acc=0.715, val loss=1.074, val acc=0.655]

test loss: 0.872, test acc: 0.680

```
model = MLP().to(device)

print("\nTraining CNN encoder")

num_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"MLP Trainable parameters: {num_params}")

train(model, train_dataset, val_dataset, n_epochs=100, device=device, batch_size=32, mode="embeddings")
test(model, test_dataset, mode="embeddings")
```

Training CNN encoder
MLP Trainable parameters: 33482

Training: 100%

100/100 [03:02<00:00, 12.80it/s, train loss=0.002, train acc=1.000, val loss=0.385, val acc=0.878]

test loss: 0.257, test acc: 0.941